📱 安卓 APK 容器开发需求文档 (V1.8)

一、项目背景与目标

本项目旨在开发一款面向巴西市场的 APK 马甲包容器。该客户端通过**落地页直接提供 APK 下载**，其本质是一个**高性能、防作弊、具备深度原生能力的超级 WebView 容器**（套壳 H5）。

App 需要承载复杂的网页端任务流（如：裂变分享、截图上传、视频浏览等），因此需要安卓端提供坚实的底层 API 支持，并确保极佳的设备兼容性和极低的网络延迟。

二、核心架构与系统要求

为确保 App 具备长期稳定性和较高的设备兼容率，开发必须严格遵守以下底层规范：

1. **Target API Level:** `targetSdkVersion` **必须设置为 34 (Android 14)**，`minSdkVersion` 设置为 24 及以上，确保兼容最新的安卓系统权限机制。
2. **依赖库合规:** 严禁使用长期未维护、包含已知安全漏洞的第三方 SDK。所有 AndroidX 库、Firebase 等必须使用最新稳定版。
3. **架构模式与包体控制 (⚠ 重点):** 要求建立全新的独立项目结构，**严禁冗余其他项目或历史遗留的代码**。必须采用原生 Kotlin 开发，杜绝臃肿框架，保持包体极度轻量化（**严格控制在 5MB - 10MB 以内**，越小越利于落地页转化）。
4. **动态域名配置防劫持:** 严禁将 H5 的入口 URL 硬编码（Hardcode）在本地代码中。App 启动时需先通过安全的远端接口（如 **Firebase Remote Config 远程配置**）获取最新的 H5 真实域名后再进行加载，以应对域名被污染或封锁的情况，实现云端热更新入口。

三、功能需求详述

3.1 核心 WebView 容器层 (WebView Core)

- **原生启动页与白屏防御 (Splash Screen):** App 启动时必须展示纯原生的过渡启动页（展示 App Logo）。需等待 WebView 内的 H5 首页完全渲染完毕（或接收到 JSBridge 的 `App.removeSplashScreen()` 指令）后，原生启动页方可渐隐消失，**绝对禁止出现加载过程中的白屏闪烁**。
- **物理返回键接管 (Back Navigation):** 必须拦截 Android 系统的物理返回键/侧滑返回手势。优先执行 `WebView.goBack()` 返回上一级 H5 页面；若 H5 已在首页，则提示“再按一次退出应用”，**严禁点一次直接闪退**。

- **沉浸式与全面屏适配:** 隐藏原生状态栏底色，支持状态栏透明及字体深浅色动态切换。底部需适配全面屏手势指示条 (Padding-bottom)，避免遮挡 H5 的底部操作栏。

```
在开启沉浸式的同时，必须结合 AndroidX WindowInsetsCompat 或重写根布局的 onApplyWindowInsets，手动监听键盘高度并对 WebView 施加 Padding/Margin 动态挤压，严禁仅依赖 adjustResize 属性。
```

- **UI 稳定性与输入体验 (🆕 新增核心防坑):** 必须强制 WebView 忽略系统字体大小设置（setTextZoom(100)），防止用户调大系统字体导致 H5 排版错乱。配置 windowSoftInputMode=adjustResize，确保软键盘弹起时平滑顶起输入框，绝不能遮挡 H5 底部操作按钮。
- **App 唤醒与深度链接 (Deep Linking / URL Scheme) (🆕 新增核心链路):** 配置自定义 URL Scheme（如 ganhago://）及 App Links 支持。确保已安装 App 的用户在 WhatsApp/Facebook 点击我方推广链接时，能直接唤醒 App，并将 URL 参数传递给 H5 实现深度路由跳转，拒绝跳出到外部浏览器。

```
必须在文档中明确一条需要服务端配合的跨端需求：外包在完成 App Links 配置后，必须生成对应的 assetlinks.json 文件并交给后端研发，后端必须将其部署在 H5 域名的 .well-known 目录下。
```

- **H5 权限自动放行 (WebRTC / KYC 支持) (🆕 新增):** 若 H5 侧后续需要引入真人活体检测（Liveness/KYC），安卓端必须重写 `WebChromeClient.onPermissionRequest`。当 H5 请求相机/麦克风权限时，安卓端需自动映射并请求系统原生权限，授权后立即放行 H5 请求。
- **外部浏览器路由控制 (External Browser Routing):** 除了默认的 URI Scheme 拦截外，安卓端必须提供明确的 JSBridge 接口，允许 H5 前端强制指定某些特定链接（如第三方积分墙、广告落地页、外部客服等）跳出 App，直接唤起手机系统自带的默认浏览器打开。以此避免在 WebView 内发生第三方授权拦截、Cookie 丢失或跨域追踪失败的致命问题。
- **原生文件选择器与 Android 14 适配:** 重写 `onShowFileChooser` 唤起 Android 原生的“相机/相册”底图。**必须严格适配 Android 13+ 的 `READ_MEDIA_IMAGES` 以及 Android 14 的 `READ_MEDIA_VISUAL_USER_SELECTED` （部分照片授权）权限机制**，防止在高版本手机上崩溃。处理图片压缩（保持清晰度前提下限制大小）后，将本地 URI 回传给 H5。
- **断网与异常兜底 (Error Handling):** 针对巴西部分地区弱网环境，遇到网络超时或 H5 加载失败时，**绝对禁止显示系统默认的错误页**（如 Chrome 恐龙页）。必须提供原生的“网络异常，点击重试” UI 界面，确保体验闭环。

3.2 设备指纹与反作弊安全层 (Anti-Fraud)

- **高强度唯一设备码:** 采集 Google Advertising ID (GAID) + Android ID + Widevine ID，生成无法轻易被模拟器篡改的唯一硬件标识 (Device ID)。

```
代码块
1  在 Android 13 (API 33) 及以上的系统中，Google 引入了极其严格的广告 ID 隐私限制。如果 App 没有在 AndroidManifest.xml 中显式声明特定的权限，尝试获取 GAID 时，系统将直接返回一串毫无意义的 00000000-0000...
2  应该在 AndroidManifest.xml 中补充声明 <uses-permission android:name="com.google.android.gms.permission.AD_ID"/> 权限，否则无法获取真实 GAID。
```

- **黑灰产环境检测:** 每次启动及运行时，静默检测并拦截以下高危环境：
  - 模拟器检测 (Emulator Check)。
  - 代理 / VPN 检测。
  - Root 权限检测。
  - USB 调试 / 开发者模式状态。
  - ~~**虚拟定位检测 (Mock Location) (🆕 新增):** 识别是否开启了系统级虚拟定位伪装，防范跨区或使用脚本机房刷单。~~ 【虚拟定位检测不必实现】
  - **SIM 卡识别:** 识别当前运营商 MCC/MNC（判断是否为巴西本地如 Claro/Vivo/TIM），无 SIM 卡标记为高危。

3.3 全渠道原生分享模块 (Native Share SDK)

由于 H5 纯网页唤起分享容易失败且无法携带图片，需通过安卓原生接管分享动作。

- **图文混合分享:** 接收 H5 传来的 `(text, image_url, platform)` 参数。安卓端静默下载图片到本地 Cache，调用 `Intent.ACTION_SEND` 将图片和文案**合并**发送给目标 App。
- **定向 App 唤醒支持:** WhatsApp (`com.whatsapp`)、Facebook (`com.facebook.katana`)、Instagram (`com.instagram.android`)、TikTok (`com.zhiliaoapp.musically`)、Kwai (`com.kwai.video`)、原生短信、原生 Share Sheet 兜底。
- **App 安装状态检测:** 提供接口供 H5 查询当前手机已安装的社交软件列表。

3.4 消息推送模块 (Push Notification & 巴西到达率优化)

针对巴西市场低端安卓机型杀后台严重的情况，推送模块必须采用以下技术与策略方案：

- **底层通道与运营封装:** 底层必须集成 **Firebase Cloud Messaging (FCM)** 确保 GMS 长连接，上层**强烈要求接入 OneSignal SDK**，以便运营团队通过可视化后台进行自动化旅程 (Automated Journeys) 配置和高转化推送。

- **强制使用 Notification Message:** 为绕过三星、摩托罗拉等低端机的魔改省电强杀机制，**严禁仅依赖 Data Message（数据透传消息）**。必须规范使用 **Notification Message（通知消息）**，确保 App 在后台被杀死的情况下，通知仍能由底层 Google Play Services 接管并弹出。
- **动态权限申请策略 (两段式授权):** 针对 Android 13+ (API 33+) 的 `POST_NOTIFICATIONS` 权限，采用两阶段引导策略：
  a. **预授权引导:** 首次触发时，先展示由 H5 渲染的“预授权弹窗”（如：“开启通知每天领福利”），用户点击同意后，安卓端再拉起系统真实的权限请求框，大幅提升首日授权率。
  b. **关键节点二次召回:** 当用户“提交首次提现”等关键节点时，若通知权限为关闭状态，再次引导用户并**直接跳转至系统应用设置页**让其手动开启。
- **点击透传与深度路由:** 用户点击系统通知栏弹窗后，App 唤醒并解析 Payload 附加数据，通过 JSBridge 将参数传递给 H5 实现页面跳转。

3.5 性能优化与缓存模块 (Caching)

- **静态资源本地化与实时替换:** 重写 `shouldInterceptRequest`，对高频加载的 JS、CSS、背景图、金币图标等进行本地拦截和缓存，实现页面秒开。**必须具备缓存失效与实时更新机制**（如比对前端静态资源文件的 Hash 值或 ETag），确保 H5 发布新版后，安卓端能静默替换旧缓存，杜绝页面样式错乱。

代码块

```
“构造 WebResourceResponse 时，必须根据文件后缀动态设置极其准确的 MIME Type（如 text/javascript、text/css），严禁传 null 或通配符，否则会被 WebView 内核拦截执行。”
```

- **视频边下边播与包体瘦身策略:** 针对 H5 内的视频元素（如集成 `AndroidVideoCache`），实现首次边下边播。结合巴西手机存储极小的现状，**视频总缓存上限严格控制在 50MB - 100MB 之间（可设置为100MB）**，并采用 LRU 算法自动清理旧文件，严防 App 异常膨胀导致被卸载。

3.6 应用内静默更新模块 (In-App Update)

- 本应用通过落地页分发，必须具备独立、强悍的更新能力。
- 启动时请求服务端版本接口，遇重大更新时直接在 App 内弹窗强制/静默下载新版 APK，调用原生系统安装器执行覆盖安装（必须处理 Android 8.0+ 的“允许安装未知来源”权限申请，**及 Android 14 对后台安装的新安全限制**）。

3.7 数据归因与 Firebase 埋点模块 (Event Tracking)

- **落地页下载归因 (Out-of-Store Attribution):** 由于脱离了 Google Play，必须解决广告买量与裂变的归因问题。要求支持多渠道打包方案（如Walle/VasDolly）允许在 APK 写入渠道标识；或**在首次启动时静默读取系统剪贴板**（提取落地页写入的参数），以便 H5 精准识别拉新来源。

- **客户端原生埋点 (APK端):** 深度集成 Firebase Analytics。安卓端**仅需处理**设备与生命周期层面的打点：
  - `install_app` (App 安装激活)
  - `first_open_app` (首次打开)
  - `push_permission_status` (通知权限授权结果)

  代码块

  ```
  针对 App 的激活和首次打开，严禁安卓端手动打点，直接依赖 Firebase SDK 的自动收集机制即可；安卓端仅需手动上报那些非标准的自定义事件（如文档中的 push_permission_status 通知权限状态 ()）
  ```

- **业务埋点隔离 (H5 / 服务端):** 用户注册、完成任务、提现等高频变化的业务转化节点，**一律不在安卓端打点**，统一由 H5 前端对接数据中台完成，确保业务逻辑高频迭代时安卓端无需发版。

四、H5 与 Android 交互协议 (JSBridge API)

开发需采用成熟的 JSBridge 方案，**必须提供基于 Promise 或 Callback ID 的标准异步回调机制**，确保 H5 能准确接收安卓端的执行结果。核心接口清单如下：

API 方法名	功能描述	返回 / 入参
App.getDeviceInfo()	获取设备指纹及风控环境	返回 JSON: DeviceID、网络状态、模拟器标识等
App.getChannelInfo()	获取 APK 安装渠道或归因参数	返回 String: 渠道号或剪贴板识别的参数
App.getInstalledSocialApps()	获取本机已安装的社交软件	返回 Array: ['whatsapp', 'facebook', ...]
App.requestPushPermission()	唤起系统权限申请或跳转设置页	返回 Boolean: 当前权限状态
App.shareTo(platform, text, imgUrl)	触发指定平台的图文原生分享	入参: 平台标识、文本、图片网络地址
App.copyToClipboard(text)	调用原生剪贴板复制文本	入参: 需复制的字符串
App.vibrate(type)	触发手机震动 (短震/长震)	增强 H5 点击任务的物理反馈爽感
App.saveImageToGallery(url)	保存网络图片到本地相册	需请求存储权限 (适配 Android 13+) 对于 Android 10+，严禁申请任何权限，直接使用 MediaStore 写入；仅针对 Android 9 及以下版本，才需要申请旧版的 WRITE_EXTERNAL_STORAGE 权限。
App.removeSplashScreen()	移除原生的冷启动过渡白屏	H5 首屏挂载完毕后主动调用
App.onAppLifecycle(state)	前后台状态同步监听	安卓主动触发！入参: foreground/background。解决用户切去 WhatsApp 后切回 App 时，H5 需要刷新状态的痛点。
App.openUrlExternally(url)	强制跳出 WebView 调起系统外置浏览器	入参: URL 链接。用于处理第三方积分墙、广告落地页等需跳出 App 的场景

五、代码安全与混淆规范

1. **代码混淆 (Proguard/R8):** 必须开启最高级别的代码混淆，隐藏核心 API 接口和签名密钥。
2. **防关联与矩阵支持:** 项目结构需支持通过 Gradle 脚本或配置表便捷替换包名 (Package Name)、签名证书 (Keystore)、资源前缀和 UI 主题色，以支持未来的马甲包矩阵生产。
3. **Play Protect 风险规避 :** 必须采用 V2/V3 方案对 APK 进行完整规范签名，避免因签名缺陷被 Google Play Protect 直接拦截或报毒。

六、交付标准与验收要求

外包团队在完成开发后，必须提交以下成果：

1. **完整且开源的源代码:** 交付 100% 无任何加密、无非必要第三方闭源 SDK 的完整 Android Studio 工程源码。确保我方内部技术人员可以顺利编译打包。
2. **测试与上线级质量:** 交付的 APK 必须经过真机测试（涵盖 Android 9.0 至 Android 14 常见机型），无白屏、闪退、内存泄漏等严重 Bug。各项 JSBridge 交互流畅无阻塞。

3. **技术文档:** 提供一份简洁的 Readme 文档，说明编译环境要求、第三方 Key（如 FCM/OneSignal）的配置及替换位置、以及 JSBridge 的调用 Demo。

七、需求答疑-深度细节

此部分为执行细节，逻辑参考。

1、集成播放器可能会造成包体积过大。

这里背景未补充清晰，业务背景：我们在H5开发了一个仿tiktok的视频流，服务端会爬取tiktok、youtube、kwai、vimeo的视频链接在线解析播放，不是mp4或其他视频格式，另外部分原创的广告视频是mp4格式（或其他视频格式）。

- **总结如下：**
  视频来源：1）tiktok、youtube、kwai、vimeo的视频链接由H5在线解析播放；2）原创广告视频云链接
- **需要解决的需求：**

代码块

```
# 马甲包 APK 视频流开发需求与技术复盘纪要 (临时文档)

## 一、 核心业务架构与背景
* **业务场景**: 纯 H5 实现仿 TikTok 视频流页面。
* **视频来源**:
  1. TikTok、YouTube、Kwai、Vimeo 等外部链接（H5 内部直接解析播放）。
  2. 原创 MP4 广告视频云端链接。
* **核心限制**: APK 包体积必须严格控制在 **5-10M** 以内。
* **架构共识**: **不做重型原生播放器**。利用原生 WebView 作为轻量级高性能容器，由 H5 承载所有视频流媒体的缓冲、播放与解析。针对 MP4 广告的本地缓存策略，由前端或服务端利用 HTTP 强缓存机制解决，原生端不介入缓存逻辑。

## 二、 核心问题与解决方案要点清单 (Android 研发执行规范)

### 1. 解决视频卡顿与黑屏问题
* **核心问题**: 默认 WebView 渲染性能不足。
* **解决方案 (开启硬件加速)**:
  * 在 `AndroidManifest.xml` 中对应的 Activity 添加 `android:hardwareAccelerated="true"`。
  * 或在代码中设置: `webView.setLayerType(View.LAYER_TYPE_HARDWARE, null);`。

### 2. 解决视频无法自动连播问题
* **核心问题**: 系统默认拦截非用户主动点击的媒体播放行为。
* **解决方案 (关闭手势播放限制)**:
  * 添加配置允许代码自动控制播放: `webSettings.setMediaPlaybackRequiresUserGesture(false);`。

```

3. 解决全屏界面的沉浸感问题
* **核心问题**：原生状态栏导致仿 TikTok 页面顶部存在割裂感。
* **解决方案（开启沉浸式状态栏）**：
  * 在 Activity 中将状态栏设置为透明（Transparent），并使得 WebView 内容延伸至状态栏底部（沉浸式布局）。

4. 解决后台持续播放的幽灵声音 Bug
* **核心问题**：App 退至后台或锁屏时，WebView 内的视频音频未停止。
* **解决方案（绑定生命周期）**：
  * `onPause` 时调用 `webView.onPause()`。
  * `onResume` 时调用 `webView.onResume()`。

5. 解决第三方视频链接解析失败问题
* **核心问题**：H5 解析外部视频需要本地缓存权限，且涉及 HTTP/HTTPS 跨域混合请求。
* **解决方案（开启存储与混合加载）**：
  * 开启 DOM 存储：`webSettings.setDomStorageEnabled(true);`。
  * 允许混合加载：`webSettings.setMixedContentMode(WebSettings.MIXED_CONTENT_ALWAYS_ALLOW);`。

6. 解决无限列表导致的 OOM 闪退问题（严重 Bug 防御）
* **核心问题**：反复进出视频流页面导致 WebView 内存持续泄漏，最终撑爆内存。
* **解决方案（安全销毁机制）**：
  * 在 Activity 的 `onDestroy` 中严格按顺序执行：将 WebView 从父容器移除 -> `webView.clearHistory()` -> `webView.destroy()`。

7. 解决第三方视频播放器状态异常问题（兼容兜底）
* **核心问题**：系统默认禁止第三方 Cookie，可能导致 YouTube、Vimeo 等外部播放器的防盗链或风控拦截，出现视频加载失败或一直转圈。
* **解决方案（允许第三方 Cookie）**：
  * 添加配置：`CookieManager.getInstance().setAcceptThirdPartyCookies(webView, true);`。

8. 解决断网状态下暴露“马甲包”身份问题（UI 兜底）
* **核心问题**：H5 首页（HTML文件）首次加载失败或完全断网时，WebView 默认弹出带有错误码的系统网页错误页，直接打破原生应用伪装。
* **解决方案（原生断网拦截）**：
  * 重写 `WebViewClient` 的 `onReceivedError` 和 `onReceivedHttpError`，拦截系统默认错误页，展示原生设计的“网络开小差了，点击重试”占位图或 View。

9. 解决首次启动时的“白屏”时间差问题（体验兜底）
* **核心问题**：从 App 启动到 H5 引擎初始化并渲染出首屏，存在物理时间差，会导致屏幕出现短暂白屏闪烁。
* **解决方案（平滑过渡）**：
  * 原生端启动时立即展示与 H5 首屏一致的原生占位图（Splash Screen）。H5 首屏核心内容渲染完毕后，通过 JSBridge 通知原生端撤销占位图，实现无缝视觉衔接。

2、SIM卡和虚拟定位监测做到什么程度?

只需要调用系统基础API获取信息即可

代码块

```
# Android 端基础反作弊数据采集需求（研发执行单）

**开发目标**：不需要做 100% 的拦截防御。只需要调用系统基础 API 获取以下 3 类信息，并静默透传给后端即可。后端的风控策略会自行根据这些值做交叉校验。

## 一、 核心信息采集需求（Android 研发执行）

**1. 获取 SIM 卡状态信息**
* **是否插卡**：调用 `TelephonyManager.getSimState()`，判断当前手机是否真实插入了 SIM 卡。
* **国家代码**：调用 `TelephonyManager.getSimCountryIso()` 或 `getNetworkCountryIso()`，获取当前 SIM 卡的所属国家代码（例如巴西是 `br`）。

**2. 获取虚拟定位（Mock Location）标识**
* **定位伪装状态**：在获取 GPS 定位的 Location 对象时，直接调用 `isFromMockProvider()`（Android 12 以下）或 `isMock()`（Android 12 及以上），返回 `true` 或 `false`。（注意：若用户未授权定位权限，直接捕获异常并默认返回 `false`，严禁导致闪退）。

**3. 获取网络环境信息**
* **VPN 状态**：通过 `ConnectivityManager` 检查当前网络的 `NetworkCapabilities` 是否包含 `TRANSPORT_VPN`，判断是否开启了代理或 VPN，返回 `true` 或 `false`。

## 二、 JSBridge 接口设计（前端与 Android 对接）

原生 Android 端需封装以下接口，供 H5 前端随时调用。

* **方法名称约定**：`window.JSBridge.getDeviceRiskInfo()`
* **返回数据结构（JSON 字符串格式）**：

```json
{
  "has_sim_card": true,       // true: 已插卡; false: 未插卡或未知
  "sim_country": "br",        // 具体的国家代码，无卡或获取失败时返回 ""
  "is_mock_location": false,  // true: 使用了虚拟定位; false: 真实定位或无权限
  "is_vpn": false             // true: 开启了VPN/代理; false: 真实网络
}
```
```

3、静态资源本地化，静默替换旧缓存

这一块

代码块

```
### 3.5 性能优化与静态资源本地化 (Caching & Performance)


* **核心静态资源本地拦截 (UI秒开兜底)**
    * **执行标准**：安卓端必须重写 WebView 的 `shouldInterceptRequest` 方法。专门针对 H5 的核心框架资源（如高频加载的 `.js`、`.css`、底部导航栏图标、公共背景图等）进行请求拦截。
    安卓端在构造本地文件的 WebResourceResponse 时，**必须手动注入跨域 Header，即：response.getResponseHeaders().put("Access-Control-Allow-Origin", "*");**

    * **效果要求**：当 H5 发起这些指定静态资源的请求时，安卓底层直接封装并返回存放在本地沙盒目录中的对应文件，不走网络请求。确保在弱网状态下，App 的基础 UI 骨架也能实现毫秒级瞬间渲染，绝对杜绝页面白屏。

* **资源包静默热更新机制 (动态覆盖)**
    * **触发时机**：App 每次冷启动（处于原生 Splash Screen 过渡期）时。
    * **执行标准**：安卓端在后台静默请求服务端的“静态资源版本校验接口”，比对本地资源与线上资源的 Hash 值（或 ETag/版本号）。若前端发布了新版本，安卓端需静默下载最新的资源压缩包（Zip），解压并完整覆盖本地旧目录。整个更新过程必须对用户无感，确保 H5 样式实时同步且不错乱。
    **热更新的文件替换策略：**下载的 Zip 必须先解压到一个**临时目录**（如 temp_res）。解压完成后，采用 **原子重命名 (Atomic Rename)** 操作替换旧文件夹；或者更稳妥的做法是：**“本次冷启动下载解压，下一次冷启动才生效应用”**，绝对禁止在 WebView 渲染期间暴力覆盖同名文件。

* **视频播放与包体瘦身红线 (严禁原生缓存)**
    * **架构约束**：**彻底废除**通过集成 `AndroidVideoCache` 等第三方重型库来进行视频缓存的方案。
    * **执行标准**：安卓原生端绝对不干预、不接管任何视频流媒体（TikTok/YouTube Iframe 或 MP4）的下载与本地缓存池管理。视频的边下边播、网络缓冲与多媒体内存释放，100% 交由系统自带的 WebView 底层引擎与 H5 前端代码自行消化处理。
```

4、消息推送模块-答疑

代码块

```
### 3.4 消息推送通道与路由分发模块 (Push Notification)

**核心定位**：安卓端在此模块中**仅作为透明的底层通道**，无需关心具体的业务触发逻辑（如任务下发、日常召回等）。业务逻辑全权由后端与 H5 闭环，安卓端仅需完成 SDK 集成、Token 获取及点击后的深度路由分发。
```

1. SDK 集成与 Token 管理
* **底层通道**：必须集成 Firebase Cloud Messaging (FCM) 确保 GMS 基础长连接。上层统一接入 **OneSignal SDK**。
* **Token 上报接口**：提供 JSBridge 方法（如 `App.getPushToken()`），供 H5 在用户登录时获取当前的 OneSignal Player ID / FCM Token，以便 H5 提交给服务端。

2. 消息到达率兜底策略（极度重要）
* **强制使用 Notification Message**：为绕过巴西市场低端机型（如三星、摩托罗拉）的魔改省电强杀机制，**严禁仅依赖 Data Message（数据透传消息）**。必须规范使用 Notification Message，确保 App 在后台被完全杀死的情况下，通知仍能由系统底层的 Google Play Services 唤起并弹出。

3. 动态权限申请策略（两段式授权）
针对 Android 13+ (API 33+) 的 `POST_NOTIFICATIONS` 权限，采用两阶段引导策略：
* **预授权引导**：首次触发时，先展示由 H5 渲染的“预授权弹窗”。用户点击同意后，安卓端再拉起系统真实的权限请求框，以提升授权率。
* **二次召回**：当用户进行关键业务（如提现）时，若通知权限为关闭状态，再次引导用户并直接跳转至系统应用设置页手动开启。

4. 消息格式与深度路由分发标准（Click Routing & 时序兜底）
* **协议约定**：服务端下发的所有推送消息中，其附加数据（Additional Data / Payload）必须包含统一的路由字段（约定 Key 为：`jump_url`）。
* **点击分发机制**：用户点击系统通知栏弹窗唤醒 App 后，安卓端必须解析 Payload，提取 `jump_url` 的参数值。
* **H5 传参时序兜底（防丢失机制）**：
  * **热启动状态**（App 在后台存活）：直接通过主动调用前端全局方法（例如：`window.JSBridge.onPushClicked(jump_url)`）将路由传递给 H5。
  * **冷启动状态**（App 被彻底杀死）：安卓端拿到 `jump_url` 后，**必须先将该参数暂存在原生内存中**。绝对不能立刻调用 JSBridge。必须等待 H5 首页完全渲染完毕（例如在收到 H5 调用的 `App.removeSplashScreen()` 指令后），再将暂存的 `jump_url` 传递给 H5，执行路由跳转，随后清空该暂存值。

5、应用内静默更新模块-答疑

代码块

3.6 应用内自升级模块（In-App Update）

[cite_start]**核心定位**：本应用必须具备基础且通畅的 APK 覆盖更新能力 [cite: 57] [cite_start]。采用最成熟的“接口核对 -> 系统 DownloadManager 下载 -> 引导安装”流程 [cite: 58]。要求代码极简，严防流量浪费与文件冲突。

1. 版本核对与冷启动策略
* [cite_start]**核对机制**：*每次开屏 Splash Screen 阶段，Android 端主动请求服务端“版本配置接口”获取最新 `versionCode` [cite: 58]。*

```
* [cite_start]**严格超时兜底（防假死）**：网络请求必须设置**最大超时时间 3-5 秒**[cite: 27][cite_start]。若超时或无网，必须捕捉异常并立刻放行进入 H5，严禁开屏假死[cite: 13, 14, 27]。

#### 2. 更新交互规范
* **强制更新（Force Update）**：
    * [cite_start]弹出**不可取消**的 Dialog（彻底屏蔽返回键与外部点击）[cite: 58]。展示 0%-100% 进度条，失败展示“重试”按钮。
    * **防绕过**：若用户拉起系统安装界面后点击“取消”并退回 App，原生端必须在 `onResume` 中重新弹出该强制更新 Dialog。
* **非强制更新（Optional Update）**：
    * 弹出可取消 Dialog，用户确认后后台执行下载。
    * **防重复提交**：点击更新后，若检测到相同版本号的下载任务已在运行，严禁重复开启新任务。
    * **立即安装**：收到下载成功广播后，只要 App 在前台，立刻强制拉起系统安装界面。

#### 3. 文件管理与防死锁规则（核心防坑）
* **动态命名与历史清理**：
    * 每次触发新版本下载前，必须先清空下载目录下的所有历史 `.apk` 缓存文件。
    * 新下载的文件名必须动态拼上目标版本号（例如：`update_v${versionCode}.apk`）。
* **防重复下载（省流量）**：只有当精准匹配目标版本号的 APK（如 `update_v102.apk`）已完整存在于本地时，才跳过下载直接拉起安装。

#### 4. 系统底层适配与矩阵支持（针对 Android 14 适配）
* [cite_start]**后台唤起拦截兜底**：若下载完成时 App 在后台，严禁直接唤起安装[cite: 58]。需标记 `isUpdateReady=true`，等切回 App 触发 `onResume` 时，重置标记为 `false` 后再拉起安装界面。
* **路径与 FileProvider 动态配置**：
    * [cite_start]`DownloadManager` 必须强制存入外部私有目录（`getExternalFilesDir`）[cite: 55]。
    * [cite_start]`AndroidManifest.xml` 的 authority 必须动态配置为 `android:authorities="${applicationId}.fileprovider"`，严禁硬编码包名以支持马甲包矩阵[cite: 9, 72]。
* **权限与 Intent 规范**：
    * [cite_start]声明 `REQUEST_INSTALL_PACKAGES` 权限[cite: 58][cite_start]。拉起 Intent 必须附加 `FLAG_ACTIVITY_NEW_TASK` 与 `FLAG_GRANT_READ_URI_PERMISSION`[cite: 58, 69]。
    * [cite_start]Type 显式指定为 `application/vnd.android.package-archive`[cite: 58]。
```

6.

代码块

```
# 3.1 核心 WebView 容器引擎层（WebView Core）研发执行规范

```

[cite_start]**核心定位**：本项目不开发重型原生播放器 [cite: 178]。WebView 既是网页容器，又是高频视频流媒体的承载引擎。开发必须严格执行以下配置，以解决视频黑屏、启动白屏与 H5 交互断层问题。

一、 视频流引擎与底层 Settings 配置 (核心基础)

为支撑 H5 内的视频流（TikTok/YouTube Iframe 及 MP4）流畅解析与播放，初始化 WebView 时必须强制开启以下设置：

* [cite_start]**解绑手势限制 (防无法连播)**：配置 `webSettings.setMediaPlaybackRequiresUserGesture(false);`，允许 H5 代码自动触发视频播放 [cite: 201, 202]。
* **跨域与防盗链兜底 (防外部视频黑屏)**：
    * [cite_start]开启 DOM 存储：`webSettings.setDomStorageEnabled(true);` [cite: 231]。
    * [cite_start]允许混合加载：配置 `MIXED_CONTENT_ALWAYS_ALLOW` [cite: 232]。
    * [cite_start]允许第三方 Cookie：配置 `CookieManager.getInstance().setAcceptThirdPartyCookies(webView, true);`，防止 YouTube 等外部播放器的风控拦截 [cite: 248, 249]。
* [cite_start]**生命周期绑定 (防“幽灵声音”)**：WebView 生命周期必须与 Activity 绑定。`onPause` 时调用 `webView.onPause()`，`onResume` 时调用 `webView.onResume()` [cite: 219, 220, 222]。
* [cite_start]**硬件加速与内存回收**：确保开启硬件加速 (`android:hardwareAccelerated="true"`) [cite: 189, 191][cite_start]。在 `onDestroy` 中严格执行销毁顺序：移除 WebView -> `clearHistory()` -> `destroy()`，防 OOM 闪退 [cite: 240]。

二、 启动过渡与 UI 稳定性控制

* [cite_start]**原生 Splash 移交权**：App 冷启动必须展示原生过渡页（Logo） [cite: 92][cite_start]。**必须且只能**等待接收到前端 JSBridge 的 `App.removeSplashScreen()` 指令后，原生启动页方可渐隐消失，严禁白屏闪烁 [cite: 92, 93, 265]。
* **UI 防变形强制策略**：
    * [cite_start]**禁随系统缩放**：强制配置 `setTextZoom(100)`，防止用户调大系统字体导致 H5 页面错乱 [cite: 96, 97]。
    * [cite_start]**键盘遮挡防御**：配置 `windowSoftInputMode=adjustResize`，确保软键盘平滑顶起输入框，不遮挡操作按钮 [cite: 98]。
* [cite_start]**物理按键接管 (明确兜底逻辑)**：拦截物理返回键/侧滑手势。优先执行 `WebView.goBack()`；若系统判定无法 goBack（即 H5 已在首页），则必须 Toast 提示“再按一次退出应用”，严禁点一次直接闪退 [cite: 94]。

三、 任务文件上传与权限闭环 (业务关键)

* [cite_start]**标准文件选取与回传 (消除代码歧义)**：重写 `WebChromeClient.onShowFileChooser` 唤起系统原生底图 [cite: 104][cite_start]。获取图片并进行原生压缩后，**安卓端必须将压缩后的新文件封装为 `Uri`，通过该方法自带的 `ValueCallback<Uri[]>` 对象回调给 WebView 底层** [cite: 105]。*（注意：仅以此标准回调方式传值，方可确保 H5 的 `<input type="file">` 标签能正常读取文件并上传，严禁私自通过 JSBridge 传本地绝对路径字符串。）*

```
* [cite_start]**权限动态映射**：当 H5 请求相机/麦克风时（支持 KYC 活体），安卓端需重写 `onPermissionRequest` 自动向系统申请原生权限，系统授权后立即放行 H5 请求 [cite: 101, 102][cite_start]。同时需严格适配 Android 13/14 的媒体权限（`READ_MEDIA_IMAGES` / `READ_MEDIA_VISUAL_USER_SELECTED`）以防崩溃 [cite: 104, 105]。

## 四、 外部路由拦截与断网兜底
* [cite_start]**App Links 深度唤醒**：配置自定义 URL Scheme（如 `ganhago://`）。当在 WhatsApp/Facebook 点击推广链接时，需直接唤醒 App，并将 URL 参数传递给 H5 实现深度跳转，拒绝跳出 [cite: 99, 100]。
* [cite_start]**第三方页面强制跳出**：提供 JSBridge 接口 `App.openUrlExternally(url)`。允许 H5 强制将特定链接（如第三方积分墙、广告落地页）跳出 App，唤起手机系统默认浏览器打开 [cite: 103]。
* [cite_start]**断网无缝重试**：遇到网络超时或 H5 加载失败，绝对禁止显示系统默认错误页 [cite: 106, 254][cite_start]。必须提供原生“网络异常，点击重试” UI [cite: 106, 258]。
```

6、 全渠道原生分享模块 (Native Share SDK)

这个模块我们要实现的是将webview业务端的图文分享内容用当前支持的最便捷最友好的方式，提供给用户一件分享到各个渠道。

代码块

```
# 3.3 全渠道原生分享模块 (Native Share SDK) 研发执行规范

[cite_start]**核心定位**：由于 H5 纯网页唤起分享容易失败且无法携带图片，需通过安卓原生接管分享动作 [cite: 38][cite_start]。为严格控制包体积，**绝对禁止接入任何官方分享 SDK**。所有分享必须纯靠 Android 底层 `Intent.ACTION_SEND` 机制实现 [cite: 39]。

## 一、 JSBridge 接口定义与极简交互
* [cite_start]**获取已安装列表**：提供接口供 H5 查询当前手机已安装的社交软件列表 [cite: 43]。安卓端通过 `PackageManager` 检查目标包名是否存在，返回 Array 给 H5（如 `['whatsapp', 'facebook']`）。H5 依此决定展示哪些按钮。
* [cite_start]**触发分享接口**：接收 H5 传来的 (text, image_url, platform) 参数 [cite: 39]。
    * **业务解绑声明**：由于业务采用“截图审核”模式，该 JSBridge 接口**无需**承载复杂的成功/失败状态回调来用于发奖。安卓端仅需负责“尽最大努力拉起目标 App”即可，无需监听发送结果。

## 二、 纯文本与图文混合的流转隔离（防闪退核心）
必须根据 `image_url` 是否为空，执行两套完全独立的 Intent 组装逻辑：

* **分支 A：纯文本分享（`image_url` 为空）**
    * 跳过图片下载。直接组装 `Intent.ACTION_SEND`，强制设置 `intent.setType("text/plain")`，将 `text` 放入 `Intent.EXTRA_TEXT`。
```

```
15
16  * **分支 B: 图文混合分享 (`image_url` 不为空)**
17      1. [cite_start]**沙盒静默下载**: 安卓端静默下载图片到本地 Cache [cite: 39]。**每次下载前必须清空该目录下的历史分享图，防 OOM**。
18      2. **FileProvider 转换**: 通过 `FileProvider` 将图片转为 `content://` URI，严禁暴露 `file://`。
19      3. [cite_start]**组装 Intent**: 调用 `Intent.ACTION_SEND` 将图片和文案合并发送给目标 App [cite: 39]。设置 `intent.setType("image/*")`，附加 `FLAG_GRANT_READ_URI_PERMISSION`，将 URI 放入 `EXTRA_STREAM`。
20
21  ## 三、 定向 App 唤醒支持与兜底策略 (跨端排坑)
22  [cite_start]安卓端必须针对 H5 传来的 `platform` 严格执行以下策略。支持 WhatsApp、Facebook、Instagram、TikTok、Kwai、原生短信、原生 Share Sheet 兜底 [cite: 40, 41, 42]。
23  **警告：所有定向包名的 `startActivity` 必须被 `try-catch` 包裹，捕获异常后立刻回落到“原生 Share Sheet”。**
24
25  ### 1. WhatsApp (`com.whatsapp`)
26  * **图文完美支持**: 设置 `intent.setPackage("com.whatsapp")`。同时放入 `EXTRA_STREAM` (图片) 和 `EXTRA_TEXT` (文案)，WhatsApp 会自动将文案作为图片说明。
27
28  ### 2. Facebook (`com.facebook.katana`) & Instagram (`com.instagram.android`)
29  * **政策封杀与剪贴板兜底**: Meta 官方严禁通过 API 预填文案，同时传入图文会强行丢弃文案，导致用户无法截到带有拉新链接的图。
30  * **强制执行 (极度重要)**:
31    1. 拉起前，**安卓端必须在后台将 `text` 强制复制到系统剪贴板**。
32    2. 屏幕中央 Toast 提示：“已复制分享链接与文案，请在发帖时粘贴”。
33    3. Intent 仅携带图片 URI (`EXTRA_STREAM`) 拉起目标 App，由用户自行粘贴文案后截图。
34
35  ### 3. TikTok (`com.zhiliaoapp.musically`) & Kwai (`com.kwai.video`)
36  * **短视频兼容防雷**: 这两大短视频平台对外部静态图文的接收支持极差。依赖上文的 `try-catch` 机制，若抛出不支持异常，立刻静默走 Share Sheet 兜底逻辑。
37
38  ### 4. 原生短信 (SMS)
39  * **防彩信高额资费**: **强制忽略图片参数**。使用 `intent.setAction(Intent.ACTION_SENDTO)` 和 `intent.setData(Uri.parse("smsto:"))`，将文案直接塞入 `sms_body` 字段，实现纯文本安全发送。
40
41  ### 5. 原生 Share Sheet (原生兜底)
42  * **触发条件**: `platform` 未匹配，或定向唤醒抛出异常时。
43  * **执行**: 调用 `Intent.createChooser(intent, "Share via")`，拉起安卓系统底层的九宫格分享面板。
```

7、消息推送模块 (Push Notification & 巴西到达率优化)

代码块

```
## 四、 深度路由分发标准（防跳空与时序死锁，极度重要）
**协议约定**：服务端下发的 Payload 附加数据中，必须包含统一的路由字段（约定 Key 为：`jump_url`）。

**H5 传参时序兜底（核心防坑，外包极易翻车）**：
⚠️ **开发注意**：提取 `jump_url` 参数必须在 OneSignal SDK 的 `OSNotificationOpenedHandler` 回调中进行解析，严禁使用原生 Activity 的 Intent 拦截。

解析拿到参数后，必须严格区分以下两种 App 存活状态进行分发：

* **状态 A：热启动状态（App 在后台存活）**
    * **场景**：用户将 App 切到后台，收到推送并点击。
    * **执行**：此时 WebView 已就绪。拿到 `jump_url` 后，直接主动调用前端全局方法（如：`window.JSBridge.onPushClicked(jump_url)`）将路由传递给 H5。

* **状态 B：冷启动状态（App 被彻底杀死）**
    * **场景**：App 进程不在，收到推送并点击启动。
    * **致命漏洞**：若拿到参数后立刻调用 JSBridge，此时 H5 尚未加载完毕，参数必然丢失！
    * **强制执行兜底**：拿到 `jump_url` 后，**必须先将其暂存在原生内存的全局变量中**。绝对不能立刻调用 JSBridge。必须等待 H5 首页完全渲染完毕（即收到 `App.removeSplashScreen()` 指令后），再将暂存的 `jump_url` 传递给 H5 执行跳转，随后清空该暂存值。
```